ChatGPT

Building a Baseball Dynasty Simulator

We surveyed dev blogs, community forums, and industry analyses on what makes top-tier dynasty sims *addictive*. Successful sims treat each subsystem (prospects, trades, onboarding, etc.) not in isolation but as part of a seamless, rewarding loop. Below we break down each design area with actionable insights. (Citations show where devs or players have discussed these points.)

Prospect Development: Keeping Progress Engaging

A key pitfall in minor-league progress is *passivity*: prospects "just aging in a box." To avoid that "waiting room" feeling, give players reasons to care at each step. For example, show visible progress bars or news alerts when a farm player improves a skill. Offer choices (like specialized training or mentors) so users actively shape development. Tie performance to outcomes – e.g. if a prospect has a hot streak in AA, trigger a call-up or special achievement message. Some sims let you micro-manage drills or coaching ("You instructed Catcher Smith on arm strength, raising his arm rating"). In practice this could be implemented as simple interactive tasks or sliders that modestly affect a player's stat growth. Keep feedback frequent: show "season highlights" for your farm teams, or use historical comparisons (e.g. "No.3 prospect's current OPS matches an all-time great at that age"). In short, make each minor-league season feel like a scene in the story, not just a page of numbers.

Trade AI: Fair and Exploit-Proof Negotiation

Preventing easy trade "exploits" means giving the CPU consistent, realistic valuations and tactics. Recent sims illustrate best practices: CPU teams should be classified as **buyers vs. sellers** and should negotiate accordingly [1]. For example, teams in contention will seek proven veterans (and value their farm less), while rebuilding teams will offload high-salary vets for prospects [2]. Key factors – roster needs, contract terms, playoff chase – must all feed into the AI's offer. On the user negotiation side, developer blogs (e.g. ZenGM) advise eliminating random haggling. Instead of asking "what's missing" endlessly, the AI should evaluate the current deal's value and suggest concrete adjustments. One algorithm: determine which side currently has the edge, then add the *most impactful asset* for the disadvantaged side to even things out [3] [4]. This avoids "negotiation spamming" and makes AI responses stable and transparent. In practice, tune CPU trade logic so that if you give them extra stars, they accept without further demands, but if your offer is short, they counter with just what's needed rather than a random extra player. Together, these measures make CPU trades feel like actual GM deals – reducing abuses where a player could repeatedly game naive AI.

Realism vs. Fun: Balancing the Line

Veteran designers stress that *baseball sim* is a game first. One The Show developer noted: "We don't make realism and fun an either/or... the question is how do we accurately represent reality in a way that is fun and interesting to the user" [5]. In practice, this means allowing spectacular or disappointing seasons even if they're a bit extreme – it creates memorable moments. By contrast, a mobile indie dev emphasizes **fun >**

realism. He deliberately omitted harsh injuries (one memorable DiMaggio rooky wreck) and other features that felt unfair [6]. The lesson: offer realism to the extent it enhances player control or narrative (e.g. strategy depth), but cut features that mostly frustrate (over-frequent injuries, maddening arcane rules).

Implement this balance by *toggling* detail: let advanced players enable full rule sets or stat effects, while newcomers can turn off the hairier ones. When adding a new feature, test for hidden buffs/penalties: as Baseball Mogul's lead dev explains, any powerful new mechanic must come with a counterbalance or "else it raises league stats in an unrealistic way" [7]. For example, if you let a gun-slinger pitcher crush one pitch type, you must slightly weaken him elsewhere so overall outcomes stay plausible [7]. This symmetrical design ensures added fun features don't break realism.

Ultimately, let *rewards drive play*, not dry numbers. Build in streaks, slumps, and surprise breakout seasons even if slightly beyond "average expectation" – players crave the emotional highs of success in a coachable environment [5] [6]. If a stat feels "unfair" (e.g. 61 HR by A-Rod), that's often memorable fun. Provide tools (like sliders or difficulty levels) so purists can tighten realism and casual players can bump up the spectacle.

Dynasty Loop: Sustained Engagement Over Decades

Long-term addicts describe dynasty play as crafting an unfolding story. They recommend building a personal narrative: start with a small league (e.g. 6–8 teams), even invent a map or history, then sim a few weeks at a time [8]. This way you "see who is doing what" in early seasons and watch first record books fill out. A top tip is to do *periodic summaries*: for example, one user writes monthly "journal" entries on team progress, letting him consume the sim results like a fan [9]. Another hobbyist manually votes Hall of Fame each year and only inducts one player, forcing reflection on league history [10].

Design-wise, support these habits: provide built-in recap screens, historical leaderboards, and customizable awards. Track and display **milestones** (e.g. "new single-season HR record" or "first 20-win season") so players feel visible progress. Incorporate a Hall-of-Fame or legacy system so they can honor franchises and icons. Even simple record tables (team's best/worst stats, all-time leaders) help. The screenshots below of a vintage-style scoreboard hint at how visual milestones resonate – a dedicated "history" tab or graphic can do the same in-app [10]. Finally, ensure the game world keeps evolving: player retirements, expansion teams, rule changes, upsets and dynasties all emerge organically if simulated well. Variety in events (surprise upsets, dynasty runs, rivalries) keeps the player invested for *hundreds* of seasons.

Visualizing achievements helps retention: players stay hooked by tracking milestones (wins, awards, Hall-of-Famers). Supporting in-game history pages or leaderboards lets them see a legacy build over 20+ seasons [10] *.*

Onboarding for Non-Fans: Gentle Learning Curve

Without a giant tutorial wall, the key is **just-in-time education**. Allow new players to start with high-level tasks (set lineup, hire a coach) while hiding low-level rules. Practical approaches include turning on heavy "auto management" so the AI handles payroll, minor staff, etc. [11] . Let the user focus initially on obvious decisions (who to sign, draft, play). Provide context-sensitive tooltips for baseball jargon and stats. For example, in OOTP's new modes reviewers praise how easy it is to join and understand them (the "Perfect Draft" card mode was noted as "streamlined" and newbie-friendly) [12] . Similarly, NBA2K and other sports sims often unlock complexity over time via menus or progression.

Encourage learning by doing: pop up brief explanations on first use of features (e.g. "Salary arbitration: free agents can decline your contract after X years"). Offer a glossary or help pane (as users suggest, the official MLB site's glossaries or beginner YouTube guides can supplement). Tutorials can be optional, but the UI should be uncluttered. Community advice: *"focus on the majors, turn on auto-management, and Google baseball rules as you go"* [11] [13] . In short, expose novices to manageable chunks, use defaults liberally, and make the interface clear. (A well-placed "explain this stat" icon or a simple animated guide can go a long way.)

Procedural Player Generation: Rich Fictional Rosters

Good dynasties always feel populated by characters, real or imagined. When generating fictional players, aim for *memorable names and backgrounds*. Use name lists filtered by nationality so a Japanese prospect gets a Japanese name, etc., and avoid awkward repeats. Short "biographies" or notes can add flavor: e.g. *"Late-bloomer hurler, known for devastating forkball"* or *"Hearty slugger from small college, workhorse personality"*. Spice it up with quirks – one sim might randomly note an obsession (hot sauce in warm-ups) or a unique trait (ambidextrous thrower).

Community-driven examples show how much this matters. The quirky baseball sim **Blaseball** leaned hard into fun names and lore: players had names like *"Blood Hamburger"* or *"Nagomi Nava"* [14], and even non-baseball details (custom coffee blends "with lavender" or "cinnamon and mango") filled out the world [15]. Fans wrote *fictions and wikis by the truckload* about these characters [16]. While your sim may not go that surreal, the lesson is that an unusual name or an evocative little backstory makes a virtual player feel real. Encourage (or at least support) community storytelling: maybe allow custom nicknames or backstories for drafted players. At minimum, ensure stats and narratives align (a speedy base-stealer should *feel* fast in descriptions, a patient hitter should have good plate discipline in the writeup).

By contrast, sterile randomness turns players into interchangeable blobs. Instead, blend consistency with surprise: most fictional players should have plausible names and archetypes (pitcher, slugger, etc.), but every few years throw in a wildcard (like Blaseball's crazy fan-submitted name *"Scratch Deleuze"* [17]) to keep things fresh. Building in a light procedural "news generator" (e.g. random minor-league reports or scouting blurbs) will also help players feel like prospects are living breathing people.

Historical vs Fictional Modes: Flexibility for Fans

Opinions vary. Many players relish a *historical* mode: all the real team names, stadiums and stats create instant nostalgia and connection. Managing modern-day Mets or 1920 Yankees can be a draw, and it's educational for newcomers to see real rosters. Others prefer a *purely fictional universe*, where they set all the parameters (league size, budgets, rules) without constraints. Fictional leagues excel at empowering creativity: you can write your own "State League" lore or have your team go undefeated in Year 1. The best sims simply offer both. For a React sim, consider supporting a current-MLB roster option *and* a sandbox where everything is made-up. If resources are limited, start with a fictional default and *import real rosters* as a separate mode. That way you capture players who want authentic baseball and those who want world-building.

Mobile Dynasty UX: Simplifying the Screen

Condensing a full franchise view into a phone screen is still an open challenge. In practice, most deep sims remain PC-first. If you target mobile, prioritize clarity: use collapsible lists, filter options, and thumb-friendly controls. ZenGM's web games exemplify a mobile-aware design – their interfaces auto-scroll and hide columns as needed (you can play on a phone browser) [18]. For example, show only key stats (e.g. BAT/ERA and positional rating) in the main roster view, with detail panels on tap. Allow pinch-to-zoom on tables or break them into multiple pages (e.g. offense vs defense tabs). Use large buttons and confirm screens to avoid fat-finger errors. If space is tight, offer a "manager mode" that auto-sim certain tasks (like minor-league or training), so the player only taps when needed.

In short, test relentlessly on real phones. There's no silver bullet, but smart UI can make roster management tolerable. (For inspiration, check sports apps: some use card stacks or swipe-to-sign features.) If full freedom proves unwieldy, consider simplifying: maybe focus the phone UI on one part of the loop (scouting or trades) and direct users to desktop for full control. Above all, keep the experience snappy; slow sims on mobile kill engagement.

Key Takeaways for a 500-Hour Sim

- **Dynamic Progression:** Make every game, season, or training drill feel meaningful. Track progress visibly (stats, leaderboards, narrative events) and give players control over player growth.
- **Smart AI & Fairness:** Use well-tested algorithms for trades and negotiation (no magic "always accept X" loopholes). Let AI act in realistic team roles (buyers vs. sellers) and provide consistent feedback.
- **Balanced Realism:** Offer depth (detailed stats, realistic aging) without punishing tedium. When adding features, ensure they don't distort outcomes. Prioritize fun: allow underdog triumphs and star performances even if slightly exaggerated.
- **Long-Term Goals:** Build in reasons to keep playing past Season 5. This includes achievements (titles, career milestones), narrative events, customizable rules, and a sense of unfolding history (records, hall-of-fame). Encourage players to see their franchise legacy.
- **Gentle Learning Curve:** Don't overwhelm beginners. Provide optional tutorials, auto-management toggles, and context help. Let newbies "learn by winning" and gradually peel back complexity.
- **Memorable Roster:** Even fictional players need personality. Use distinctive names, consistent backgrounds, and small random quirks. Procedural generation should aim for both variety and plausibility to make each new player feel real.
- **Flexibility:** Support both real-world and fictional setups. Licensed rosters draw fans, but an imaginative sandbox invites creativity. If possible, let players switch modes or blend elements.
- **Mobile-Friendly Design:** If targeting phone users, simplify interfaces. Prioritize essential information, use responsive layouts, and limit finger taps needed. Mobile sims often trim features, so decide which subsystems are "core" on small screens.

In sum, the difference between a sim you play 10 hours and 500 hours is the *believability and momentum* it creates. Every mechanic should feed into a larger narrative loop, giving players both short-term rewards and long-term payoff. By applying these principles – backed by how existing games and communities talk about them [1] [10] – a React/JSX baseball sim can evolve from a spreadsheet into a living, breathing dynasty that users want to return to season after season.

Sources: Developer posts and interviews (e.g. Out of the Park dev blog [1] [2], ZenGM algorithm notes [3] [4]), player forums and Reddit discussions [12] [11] [10], and design articles on realism vs fun [5] [6] [7]. These highlight real-world trade-offs and solutions seen in top simulation games.

---

[1] [2] Out of the Park Baseball 27 Is Overhauling Trade AI to Deliver More Realistic Deadline Deals - Operation Sports
https://www.operationsports.com/out-of-the-park-baseball-27-is-overhauling-trade-ai-to-deliver-more-realistic-deadline-deals/

[3] [4] [18] Trade finding algorithm improvements « Blog « ZenGM
https://zengm.com/blog/2021/05/trade-finding-algorithm/

[5] With Baseball Stats, Trying to Synthesize Real, Fun - Kotaku
https://kotaku.com/with-baseball-stats-trying-to-synthesize-real-fun-5528856

[6] Fun VS Realism: a Game Design challenge | by Aerilys — Astonishing Sports | Medium
https://neferetheka.medium.com/fun-vs-realism-a-game-design-challenge-4d10ec46feed

7 Talking Baseball Mogul with Clay Dreslough - Part I | Battery Power
https://www.batterypower.com/2006/6/4/193840/0749

8 9 10 I don't understand the appeal of fictional leagues...but I want to. : r/OOTP
https://www.reddit.com/r/OOTP/comments/vcikyi/i_dont_understand_the_appeal_of_fictional/

11 13 Is OOTP accessible for someone who knows little about baseball ? : r/OOTP
https://www.reddit.com/r/OOTP/comments/1mbobep/is_ootp_accessible_for_someone_who_knows_little/

12 Out of the Park Baseball 22 Review - Another Year, Another Home Run
https://www.operationsports.com/out-of-the-park-baseball-22-review/

14 15 17 The Gods Play Dice: Procedural Content in Blaseball | by The Game Band | The Game, Bland | Medium
https://medium.com/the-game-bland/the-gods-play-dice-procedural-content-in-blaseball-a9487dec0310

16 Blaseball with Chris Greenbriar - by Sam Dunnewold
https://diceexploder.substack.com/p/blaseball-with-chris-greenbriar